

ÔN PHÒNG VẤN • ANGULAR 21.2.5 • REPO-UBCK-CORE-FE

Vòng đời của một Component

Angular tự gọi giúp bạn **tám cái hàm** tại tám thời điểm cố định trong đời một khối giao diện. Biết mỗi hàm chạy **lúc nào, mấy lần**, và **tại sao** — là xong 80% câu hỏi lifecycle trong phỏng vấn. Mọi ví dụ dưới đây lấy từ chính source code của repo này.

140 ngOnInit	45 ngOnDestroy	35 ngAfterViewInit	3 ngOnChanges	1 ngDoCheck	22 afterNextRender
526 computed()	65 DestroyRef				

Số lần xuất hiện thật trong thư mục `projects/`. Đọc con số này là đã thấy câu chuyện: repo dùng rất nhiều `ngOnInit/ngOnDestroy`, gần như không dùng `ngDoCheck`, và đang dịch chuyển sang nhóm API mới (màu xanh).

01 Giải thích cho người chưa biết gì

Một **component** (thành phần giao diện — một mẫu màn hình có HTML riêng, CSS riêng và code riêng, ví dụ cái nút, cái bảng, cái dropdown) không tồn tại mãi mãi. Nó *sinh ra* khi người dùng mở màn hình đó, *sống* một lúc, rồi *chết* khi chuyển sang màn hình khác.

Lifecycle (vòng đời) là chuỗi các mốc đó. **Lifecycle hook** (móc vòng đời — cái "móc" để bạn treo code của mình vào) là những hàm có tên cố định: nếu bạn khai báo hàm tên đúng như vậy trong class, Angular sẽ *tự gọi* nó đúng lúc. Bạn không bao giờ gọi tay các hàm này.

VÍ VON

Coi component như một nhân viên mới. `constructor` (hàm khởi tạo – hàm chạy khi đối tượng vừa được tạo ra trong bộ nhớ) là lúc ký hợp đồng: mới có tên, chưa có bàn ghế.

`ngOnInit` là buổi onboarding: đã nhận đủ thông tin từ cấp trên, bắt đầu làm việc.

`ngAfterViewInit` là lúc bàn ghế máy tính đã lắp xong – giờ mới sờ được vào đồ đạc.

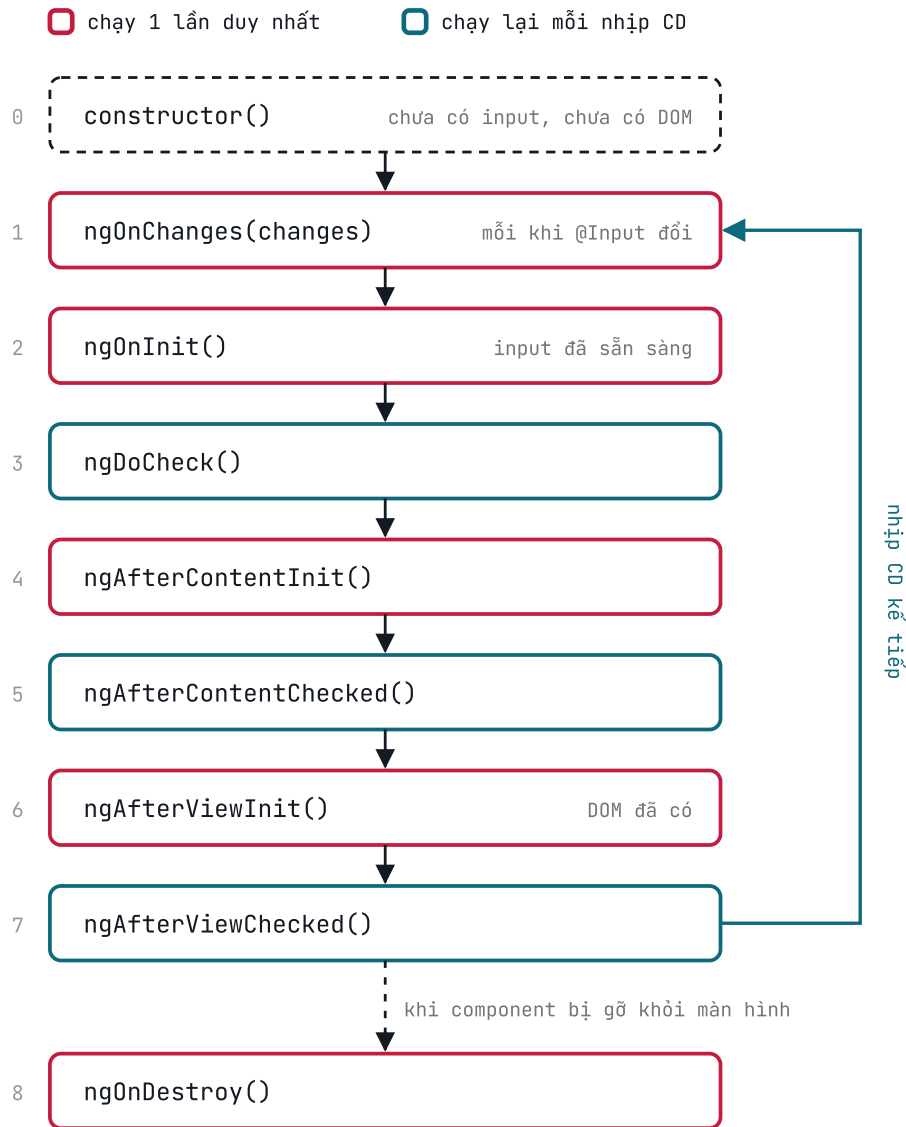
`ngOnDestroy` là lúc nghỉ việc: phải trả thẻ, tắt máy, huỷ đăng ký – nếu quên thì công ty tổn tiền mãi (đó chính là **memory leak** – rò rỉ bộ nhớ).

Ba từ nữa cần nói rõ trước khi đi tiếp:

- **Template** (khuôn mẫu) – file HTML của component, ví dụ `dropdown.component.html`.
- **DOM** (Document Object Model – cây các thẻ HTML thật mà trình duyệt đang vẽ).
Angular tạo template thành DOM.
- **Change Detection** – viết tắt **CD**, "cơ chế phát hiện thay đổi": mỗi khi có sự kiện (click, dữ liệu về từ server, hết timer...), Angular quét lại xem dữ liệu nào đổi để vẽ lại màn hình. Mỗi lần quét như vậy gọi là một *nhịp CD*. Phần lớn lifecycle hook thực chất là các móc *bên trong* một nhịp CD.

02 Thứ tự chạy – bức tranh phải thuộc lòng

Đây là câu hỏi phỏng vấn kinh điển: "*Kể thứ tự các lifecycle hook.*" Sơ đồ dưới là câu trả lời đầy đủ.



Tám hook chạy theo đúng thứ tự này. Bốn hook viền đỏ chỉ chạy một lần cho cả đời component; ba hook viền xanh (DoCheck, ContentChecked, ViewChecked) chạy lại ở *mọi* nhịp change detection – đó là lý do chúng nguy hiểm về hiệu năng.

MẸO NHỚ

Ba cặp *Init* → *Checked* đi liên nhau theo hướng “từ trong ra ngoài”: **DoCheck** (dữ liệu của chính tôi) → **Content** (nội dung người ta chiếu vào tôi) → **View** (template của tôi và các con của tôi). Cứ nhớ *data* → *content* → *view*.

03 Từng hook, và code thật trong repo

constructor() 1 LẦN CHƯA PHẢI HOOK

Không phải lifecycle hook của Angular – nó là của TypeScript/JavaScript. Chạy khi Angular `new` ra class. Lúc này **input** (dữ liệu component cha truyền xuống) *chưa* được gán, DOM *chưa* tồn tại.

DÙNG ĐỂ Chỉ **dependency injection** (DI – tiêm phụ thuộc: Angular tự đưa service vào cho bạn) qua `inject()`, và đăng ký `effect()` / `afterNextRender()` – hai thứ này bắt buộc phải gọi trong "injection context" (ngữ cảnh tiêm phụ thuộc), tức là trong constructor hoặc field initializer.

KHÔNG DÙNG ĐỂ Gọi API (giao diện lập trình – ở đây hiểu là gọi HTTP lấy dữ liệu), đọc input, đung vào DOM.

Bẫy → "Sao không gọi HTTP trong constructor cho gọn?" Vì input chưa có (bạn sẽ gọi với `undefined`), và constructor chạy cả khi **SSR** (Server-Side Rendering – kết xuất trang ở phía máy chủ) lẫn khi unit test, làm test khó mock.

+ Đào sâu: ba lý do tách bạch, và những thứ BẮT BUỘC phải nằm trong constructor

ngOnChanges(changes: SimpleChanges) MỖI LẦN INPUT ĐỔI

Chạy *trước* `ngOnInit` ở lần đầu, rồi chạy lại mỗi khi component cha gán giá trị mới cho một input. Tham số `SimpleChanges` là một object dạng `{ tênInput: { previousValue, currentValue, firstChange } }`.

QUAN TRỌNG Angular so sánh **theo tham chiếu** (reference – địa chỉ của object trong bộ nhớ), không so sánh từng field. Sửa `obj.name = 'x'` trong mảng cũ thì `ngOnChanges` *không* nổ; phải tạo object/mảng mới (**immutable** – bất biến).

TRONG REPO Chỉ 3 chỗ dùng, vì repo đã chuyển sang `input()` signal + `computed()`.

```
// projects/ui/src/custom/base/Atoms/cmm-input-switch/cmm-input-switch.component.ts
ngOnChanges(changes: SimpleChanges) {
  if (changes['defaultValue']) {
    this._value.set(this.defaultValue());
  }
}
```

```

    }
  }
}

```

Bẫy → "Dùng `input()` signal thì `ngOnChanges` còn chạy không?" – Có. Signal input vẫn được báo cáo trong `SimpleChanges`. Chứng minh ngay trong repo: `ubck-echarts-core.ts` khai báo toàn bộ input bằng `input()` nhưng vẫn cài `ngOnChanges` để lọc thay đổi.

`ngOnInit()` 1 LẦN 140 CHỖ

Chạy đúng một lần, sau lần `ngOnChanges` đầu tiên. Đây là nơi input đã có giá trị thật → chỗ khởi tạo chuẩn.

DÙNG ĐỂ Gọi HTTP, **subscribe** (đăng ký lắng nghe) một **Observable** (luồng dữ liệu phát ra nhiều lần theo thời gian, của thư viện **RxJS**), set giá trị mặc định cho form.

KHÔNG DÙNG ĐỂ Đo kích thước phần tử hay tìm `@ViewChild` – DOM chưa chắc có.

```

// projects/ubck-echarts/src/directives/ubck-echarts-core.ts:144
ngOnInit() {
  if (!isPlatformBrowser(this.platformId)) { return } // SSR: bỏ qua
  if (!window.ResizeObserver) throw new Error('please install a polyfill');

  this.resizeSub = this.resize$
    .pipe(throttleTime(100, asyncScheduler, { leading: false, trailing: true }))
    .subscribe(() => this.resize()); // ← nhớ huỷ ở ngOnDestroy
}

```

Bẫy → " `ngOnInit` chạy mấy lần?" – Một lần cho mỗi *instance*. Nhưng khi router (bộ định tuyến – thứ đổi màn hình theo URL) dừng lại cùng một component cho URL khác (ví dụ `/user/1` → `/user/2`), Angular *không* tạo instance mới ⇒ `ngOnInit` *không* chạy lại. Phải lắng nghe `route.params`.

`ngDoCheck()` MỖI NHỊP CD 1 CHỖ

Chạy ở *mọi* nhịp change detection – có thể vài trăm lần mỗi giây khi người dùng gõ phím hay rê chuột. Dùng khi bạn cần phát hiện thay đổi mà Angular không thấy được (ví dụ sửa field bên trong object cũ, hoặc trạng thái của `FormControl`).

```
// projects/ui/src/custom/validation/core/validation.directive.ts:230
ngDoCheck(): void {
  if (!this.control) return; // thoát sớm – rẻ nhất có thể
  const currentTouched = this.control.touched;
  let hasChange = false;
  if (this.previousTouched !== currentTouched) { // tự so sánh thủ công
    this.previousTouched = currentTouched; hasChange = true;
  }
  // ... dirty / valid / disabled ... rồi mới cập nhật giao diện nếu hasChange
}
```

Bắt → Đây là hook dễ giết hiệu năng nhất. Quy tắc: thoát sớm, chỉ so sánh giá trị nguyên thủy (số, chuỗi, boolean), *tuyệt đối không* gọi HTTP, không tạo object mới, không `console.log`. Cả repo chỉ có duy nhất 1 chỗ dùng – đó là con số lành mạnh.

ngAfterContentInit() 1 LẦN 0 CHỖ

Chạy sau khi phần **content** (nội dung do component cha "chiếu" vào qua `<ng-content>` – gọi là **content projection**, chiếu nội dung) đã được khởi tạo. Đây là lúc `@ContentChild` / `contentChild()` có giá trị.

Bắt → Phân biệt *content* và *view* là câu hỏi phỏng vấn rất hay gặp – xem sơ đồ mục 04.

ngAfterContentChecked() MỖI NHỊP CD

Bản "chạy lại mỗi nhịp" của hook trên. Gần như không bao giờ cần dùng trong code sản phẩm.

ngAfterViewInit() 1 LẦN 35 CHỖ

Chạy sau khi template của chính component và toàn bộ component con đã được dựng thành DOM. Đây là thời điểm sớm nhất được phép đo đạc, focus, khởi tạo thư viện bên thứ ba (chart, map, editor...).

```
// projects/ubck-echarts/src/directives/ubck-echarts-core.ts:200
ngAfterViewInit() {
```

```
if (!isPlatformBrowser(this.platformId)) { return }
this.initChartTimer = window.setTimeout(() => this.initChart());
}
```

Bẫy → Vì sao bọc trong `setTimeout` ? Hai lý do: (1) đợi trình duyệt tính xong kích thước thật của thẻ chứa biểu đồ; (2) tránh lỗi **ExpressionChangedAfterItHasBeenCheckedError** – lỗi kinh điển: bạn đổi dữ liệu *sau khi* Angular đã kiểm tra xong vòng hiện tại, khiến giá trị hiển thị khác giá trị vừa kiểm. `setTimeout` đẩy việc sang nhịp sau nên hết lỗi.

`ngAfterViewChecked()` MỖI NHỊP CD

Chạy cuối mỗi nhịp CD, sau khi view đã kiểm tra xong. Đổi state trong đây là cách nhanh nhất để tạo vòng lặp vô tận + lỗi **ExpressionChangedAfterItHasBeenChecked**.

`ngOnDestroy()` 1 LẦN 45 CHỖ

Chạy ngay trước khi Angular gỡ component khỏi DOM. Nhiệm vụ duy nhất: **dọn rác**.

Không dọn ⇒ memory leak: `setInterval` vẫn chạy, `ResizeObserver` vẫn theo dõi thẻ đã chết, subscription vẫn giữ tham chiếu tới component đã "chết" nên bộ nhớ không được thu hồi.

```
// projects/ubck-echarts/src/directives/ubck-echarts-core.ts:181
ngOnDestroy() {
  window.clearTimeout(this.initChartTimer); // 1. huỷ timer
  if (this.resizeSub) this.resizeSub.unsubscribe(); // 2. huỷ subscription
  if (this.animationFrameID) window.cancelAnimationFrame(this.animationFrameID);
  if (this.resizeOb) this.resizeOb.unobserve(this.el.nativeElement); // 3. ngắt
  if (this.loadingSub) this.loadingSub.unsubscribe();
  this.changeFilter.unobserve();
  this.dispose(); // 4. trả bộ nhớ cho ECharts
}
```

Bẫy → "Có phải Observable nào cũng cần unsubscribe?" Không. Những cái tự kết thúc (HttpClient gọi xong là complete, ActivatedRoute được router tự dọn, hoặc dùng pipe `async` trong template) thì không cần. Cái cần dọn là luồng *không bao giờ complete*: `interval`, `fromEvent`, `Subject` dùng chung trong service.

04 Content và View khác nhau chỗ nào

Câu hỏi phỏng vấn: "*Khác nhau giữa `ngAfterContentInit` và `ngAfterViewInit` ?*". Trả lời ngắn: **content** là HTML do *người khác* viết rồi nhét vào tôi; **view** là HTML do *chính tôi* viết trong template của mình.

template của CHA

```
<ubck-card>
```

```
<p>Xin chào</p>
```

↑ đây là CONTENT

```
</ubck-card>
```

chiếu vào
(projection)

template của ubck-card = VIEW

```
<div class="card">
```

```
<ng-content />
```

```
</div>
```

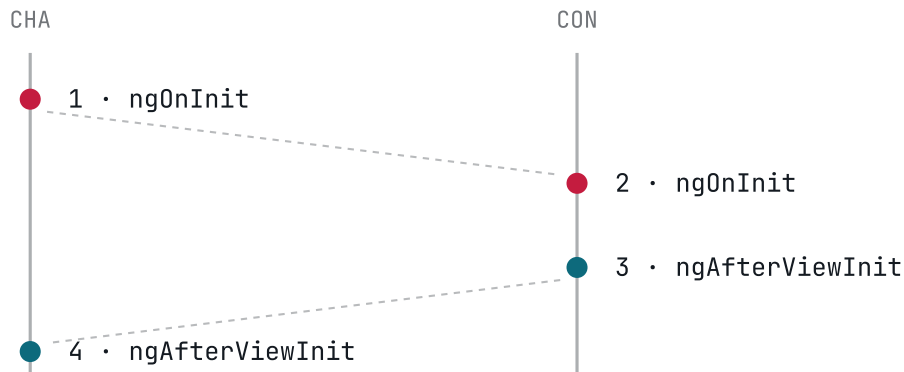
+ mọi component con bên trong

- `ngAfterContentInit()`
nội dung chiếu đã vào chỗ → đọc được `contentChild()`
- `ngAfterViewInit()`
template riêng đã thành DOM → đọc được `viewChild()`

Content đi vào qua `<ng-content>` và sẵn sàng *trước*; view của chính component sẵn sàng *sau*. Vì thế `ngAfterContentInit` luôn chạy trước `ngAfterViewInit`.

Thứ tự giữa cha và con

Câu hỏi tiếp theo thường là: "*Cha và con, ai chạy trước?*" Quy tắc: **Init đi từ cha xuống con; AfterViewInit đi từ con lên cha** – vì cha chỉ "xong view" khi tất cả con đã xong.



Khởi tạo chảy xuôi từ cha xuống con; "view đã xong" chảy ngược từ con lên cha. ngOnDestroy cũng chạy cha trước, con sau.

05 Vì sao hook lại chạy: Change Detection

Bảy trong tám hook nằm bên trong *nhịp change detection*. Vậy cái gì kích hoạt nhịp đó?

Mặc định Angular dùng **Zone.js** – một thư viện "vá" (monkey-patch) các API bất đồng bộ của trình duyệt (`setTimeout` , sự kiện click, `XMLHttpRequest` ...). Mỗi lần một trong số đó chạy xong, Zone.js báo cho Angular: "có thể có gì đó đổi, quét lại đi". Repo này vẫn đang chạy chế độ Zone.js:

```
// src/app/app.config.ts:41
provideZoneChangeDetection({ eventCoalescing: true }),
// eventCoalescing: gộp nhiều sự kiện xảy ra sát nhau thành 1 nhịp CD → đỡ tốn
```

OnPush (`ChangeDetectionStrategy.OnPush`) là cách hãm bớt: component chỉ được kiểm tra lại khi input đổi tham chiếu, khi có sự kiện phát ra từ chính nó, hoặc khi một signal nó đang đọc thay đổi. Repo dùng OnPush ở 427 chỗ – nghĩa là gần như mọi component.

ĐIỂM CỘNG KHI PHÒNG VẤN

Nói thêm về **zoneless** (chế độ không Zone.js, ổn định từ Angular 20): khi bỏ Zone.js, CD được kích hoạt bởi chính signal thay vì bởi "vá" API trình duyệt. *Các lifecycle hook không đổi*

tên, không đổi thứ tự – chỉ có tần suất các hook *Checked giảm mạnh. Repo này chưa bật zoneless, và có thể nói thẳng: đó là bước nâng cấp tiếp theo hợp lý sau khi đã phủ OnPush + signal.

06 Thời signal: những hook "kiểu mới"

Từ Angular 16 đến 21, ba nhóm API mới đã thay thế phần lớn công dụng của lifecycle hook cổ điển. Repo này là ví dụ sống: 526 chỗ dùng `computed()` nhưng chỉ 3 chỗ dùng `ngOnChanges`.

1• DestroyRef.onDestroy() – thay ngOnDestroy

DestroyRef là một service Angular thêm vào cho bạn, cho phép đăng ký hàm dọn dẹp mà *không cần* class phải implements `OnDestroy`. Nhờ vậy code dọn dẹp nằm ngay cạnh code khởi tạo, và dùng được cả trong hàm tiện ích (không phải class).

```
// projects/ui/src/custom/deferred-section/deferred-section.component.ts:39
constructor() {
  afterNextRender({ read: () => this.handleImplementWhenDomStable() });
  this.destroyRef.onDestroy(() => this.handleComponentDestroy());
}
```

Khởi tạo và dọn dẹp đứng cạnh nhau – đọc một chỗ là thấy đủ cặp.

2• afterNextRender() / afterEveryRender() – thay ngAfterViewInit

Render = trình duyệt vẽ xong lên màn hình. `afterNextRender` chạy *một lần*, sau lần vẽ kế tiếp; `afterEveryRender` (tên cũ là `afterRender`, đổi tên từ Angular 20) chạy sau *mọi* lần vẽ. Hai điểm hơn hẳn `ngAfterViewInit`:

- Chỉ chạy trên trình duyệt – tự động bỏ qua khi SSR, không cần tự viết `if (isPlatformBrowser(...))`.
- Có các "phase" (giai đoạn) `earlyRead` / `write` / `read`, giúp gom việc đọc và ghi DOM lại, tránh **layout thrashing** (đọc–ghi DOM xen kẽ khiến trình duyệt phải tính lại bố cục liên tục).

```
// projects/ui/src/utils/dropdown-helpers.ts:19
afterNextRender({
  read: () => {
    if (isDestroyed) {
      // Prevent early destroyed component causing memory leak in afterNextRender
      return;
    }
    const elementRef = elementRefFn();
    width.set(Math.round(elementRef.nativeElement.clientWidth));
    resizeObserver = new ResizeObserver(...);
  },
});
```

CHI TIẾT ĐÁNG KỂ RA KHI PHÒNG VẤN

Đoạn trên là bài học thật từ repo: component có thể bị huỷ *trước* khi lần render kế tiếp xảy ra. Vì vậy cờ `isDestroyed` được bật trong `destroyRef.onDestroy()` và kiểm tra lại ở đầu callback. Đây đúng là loại chi tiết mà người phỏng vấn muốn nghe: bạn đã gặp bug thật, không chỉ đọc tài liệu.

3. `input()` + `computed()` + `effect()` — thay `ngOnChanges` / `ngDoCheck`

Signal (tín hiệu) là ô chứa giá trị biết ai đang đọc mình; khi giá trị đổi, nó tự báo cho những chỗ phụ thuộc. `computed()` là signal dẫn xuất (tự tính lại từ signal khác). `effect()` là "hiệu ứng phụ": chạy lại mỗi khi signal nó đọc thay đổi.

<pre>// Cách cũ ngOnChanges(c: SimpleChanges) { if (c['first'] c['last']) this.fullName = ...; }</pre>	<pre>// Cách mới (repo đang dùng) fullName = computed(() => this.first() + ' ' + this.last());</pre>
---	--

RANH GIỚI CẦN NHỚ

`computed()` dùng khi bạn cần *giá trị dẫn xuất* — luôn ưu tiên. `effect()` chỉ dùng khi cần *tác động ra bên ngoài*: ghi `localStorage`, gọi thư viện thứ ba, đồng bộ với DOM. Đặt `effect()` mà chỉ để gán vào một field khác là dấu hiệu bạn nên dùng `computed()`. Muốn đọc một signal trong `effect` mà *không* đăng ký phụ thuộc vào nó thì bọc bằng

`untracked()` – repo dùng đúng kỹ thuật này ở `dropdown-checkbox.component.ts:320`.

07 Bộ câu hỏi phỏng vấn + đáp án

? Kể đủ 8 lifecycle hook theo thứ tự.

? Vì sao gọi HTTP nên đặt ở `ngOnInit` chứ không phải `constructor` ?

? `ngOnChanges` không nổ dù dữ liệu rõ ràng đã đổi – vì sao?

? Lỗi `ExpressionChangedAfterItHasBeenCheckedError` là gì, sửa thế nào?

? Có phải mọi subscription đều phải unsubscribe trong `ngOnDestroy` ?

? Signal đã thay được toàn bộ lifecycle hook chưa?

? Chuyển sang zoneless thì lifecycle có đổi không?

? Directive có lifecycle không?

08 Bảng thuật ngữ

THUẬT NGỮ

NGHĨA TIẾNG VIỆT / GIẢI THÍCH

component

Thành phần giao diện – một mẫu màn hình có HTML, CSS và logic riêng.

directive	Chỉ thị – class gắn thêm hành vi vào một thẻ HTML có sẵn, không có template riêng.
lifecycle hook	Móc vòng đời – hàm có tên quy ước, do Angular tự gọi tại một thời điểm cố định.
constructor	Hàm khởi tạo của class, chạy khi đối tượng vừa được tạo trong bộ nhớ.
template	Khuôn mẫu – file HTML của component.
DOM	Document Object Model – cây các thẻ HTML thật mà trình duyệt đang hiển thị.
render	Vẽ ra màn hình – bước trình duyệt biến DOM thành pixel.
CD	Change Detection – cơ chế Angular quét lại dữ liệu để cập nhật giao diện. Một lần quét = một "nhịp".
OnPush	Chiến lược CD chỉ kiểm tra component khi input đổi tham chiếu, có sự kiện, hoặc signal đang đọc thay đổi.
Zone.js	Thư viện "vá" các API bất đồng bộ của trình duyệt để báo cho Angular biết khi nào cần quét lại.
zoneless	Chế độ chạy không Zone.js; CD được kích hoạt bởi signal. Ổn định từ Angular 20.
input	Dữ liệu component cha truyền xuống con. Kiểu mới: <code>input()</code> trả về signal.
binding	Ràng buộc – nối dữ liệu trong class với chỗ hiển thị trong template.
content projection	Chiếu nội dung – cha viết HTML rồi nhét vào chỗ <code><ng-content></code> của con.
view	Phần DOM do template của chính component tạo ra (khác với content).
signal	Tín hiệu – ô chứa giá trị biết ai đang đọc mình và tự báo khi đổi.
computed	Signal dẫn xuất – tự tính lại từ các signal khác, có nhớ kết quả (memo hoá).

effect	Hiệu ứng phụ – hàm chạy lại mỗi khi signal nó đọc thay đổi; dùng cho tác động ra ngoài.
untracked	Đọc signal mà không đăng ký phụ thuộc vào nó bên trong effect/computed.
Observable	Luồng dữ liệu phát ra nhiều giá trị theo thời gian (thư viện RxJS).
subscribe	Đăng ký lắng nghe một Observable; ngược lại là <code>unsubscribe</code> (hủy đăng ký).
memory leak	Rò rỉ bộ nhớ – code cũ vẫn chạy/vẫn bị giữ tham chiếu sau khi component đã bị gỡ.
DI	Dependency Injection – tiêm phụ thuộc: Angular tự cấp service cho class qua <code>inject()</code> .
DestroyRef	Đối tượng cho phép đăng ký hàm dọn dẹp mà không cần implement <code>OnDestroy</code> .
SSR	Server-Side Rendering – kết xuất HTML ở phía máy chủ; lúc đó không có <code>window / document</code> .
polyfill	Đoạn mã bù tính năng còn thiếu cho trình duyệt cũ.
immutable	Bất biến – không sửa dữ liệu cũ mà tạo bản mới; giúp so sánh tham chiếu hoạt động đúng.
reference	Tham chiếu – địa chỉ của object trong bộ nhớ, thứ Angular dùng để so sánh input.
ResizeObserver	API trình duyệt theo dõi thay đổi kích thước của một phần tử.
IntersectionObserver	API trình duyệt báo khi một phần tử lọt vào vùng nhìn thấy – dùng cho lazy load.
layout thrashing	Đọc và ghi DOM xen kẽ khiến trình duyệt phải tính lại bố cục nhiều lần, gây giật.
eventCoalescing	Gộp nhiều sự kiện xảy ra sát nhau thành một nhịp CD để giảm tải.

Nguồn: source code `repo-ubck-core-fe` – Angular 21.2.5. Các dòng code trích nguyên văn kèm đường dẫn và số dòng, kiểm chứng được bằng cách mở đúng file.